

[58] A Survey on Advancing the DBMS Query Optimizer

요약

본 논문은 데이터베이스 관리 시스템(DBMS)의 쿼리 최적화기(query optimizer)의 발전을 다루는 포괄적인 서베이 논문이다. 2021년 Data Science and Engineering 저널에 발표된 이 논문은 쿼리 최적화의 세 가지 핵심 구성 요소 - 카디널리티 추정(cardinality estimation), 비용 모델(cost model), 그리고 플랜 열거(plan enumeration)에 대해 깊이 있게 분석한다.

쿼리 최적화기의 핵심 역할: - SQL 쿼리를 입력받아 효율적인 실행 계획 생성 - 여러 가능한 실행 전략 중에서 최적의 것 선택 - 데이터베이스 전체 성능에 직접적 영향

카디널리티 추정은 쿼리 최적화의 기초이다: - 각 연산 단계에서 처리되는 행의 개수를 예측 - 부정확한 추정은 최적화기를 잘못된 플랜으로 유도 - 통계 기반 방법과 머신러닝 기반 접근법의 발전 추적

비용 모델의 역할: - 각 실행 계획의 상대적 비용 계산 - CPU, I/O, 메모리 비용을 종합적으로 고려 - 정확한 비용 예측이 올바른 선택으로 이어짐

플랜 열거 알고리즘: - 모든 가능한 실행 계획을 탐색하거나 휴리스틱으로 제한 - 검색 공간의 크기 증가 문제 극복

상세분석

58.1 카디널리티 추정의 진화

카디널리티 추정은 쿼리 최적화의 가장 어려운 부분이다. 전통적 방법들:

1.1 통계 기반 방법 - 히스토그램, 스케치, 샘플링 등의 데이터 통계 활용 - 단일 속성 통계는 비교적 정확하나, 다중 속성 상관관계 파악 어려움 - 선택도(selectivity) 추정의 부정확성이 누적되는 문제

1.2 머신러닝 기반 접근 - 쿼리 특성을 학습하여 카디널리티 예측 - NeuroCard, DeepDB 등의 생성 모델 기반 방법 - 통계 방법보다 높은 정확도를 보이지만 계산 비용 증가

1.3 하이브리드 접근 - 통계 기반 방법의 빠름 + ML 기반 방법의 정확성 결합 - 적응적 학습을 통한 지속적 개선

58.2 비용 모델의 복잡성

현대 DBMS에서 비용 모델은 단순한 I/O 카운트를 넘어서는:

- CPU 사이클, 메모리 접근 패턴, 캐시 효율성
- 병렬 실행에서의 동기화 오버헤드
- 네트워크 전송 비용(분산 환경)
- 벡터 연산의 특수성(SIMD 최적화 등)

정확한 비용 예측을 위해서는: - 하드웨어 특성 반영 - 데이터 크기와 분포에 따른 동적 비용 계산 - 시스템 부하 상태 고려

58.3 플랜 열거의 도전 과제

Join order 문제에서 n 개의 테이블에 대해 가능한 순서는 $n!$ 에 비례한다:

- 작은 규모: 동적 프로그래밍으로 최적해 탐색 가능
- 중간 규모: 휴리스틱 기반 접근 필요
- 대규모: 메타휴리스틱(유전 알고리즘 등) 활용

최근 발전: - 몬테 카를로 트리 검색 기반 플래너 - 학습된 휴리스틱으로 검색 공간 제한 - 분산 환경에서의 플래닝 병렬화

58.4 벡터 검색 환경에서의 새로운 도전

전통 DBMS 최적화 이론이 벡터 검색에 직접 적용되지 않는 이유:

- **카디널리티 추정**: 벡터 거리 함수의 선택도 계산이 통계 기반으로서는 어려움
- **비용 모델**: 거리 계산 비용, 인덱스 트레이버설 비용이 차원과 데이터 크기의 함수로 비선형
- **플랜 열거**: 스칼라 조건과 벡터 조건의 혼합 필터링 순서 최적화

58.5 본 논문과의 관계

Exqutor 논문이 벡터 검색을 포함한 데이터베이스의 쿼리 실행을 분석하기 위해서는 전통 DBMS 최적화의 기본 틀을 이해해야 한다. 본 논문([58])은:

- **기본 개념 제공**: 카디널리티 추정, 비용 모델, 플랜 열거의 정의와 중요성 설명
- **최적화 문제의 구조화**: 벡터 검색 쿼리 최적화를 전통 쿼리 최적화 틀에 맞추어 분석하는 데 필수
- **성능 분석 기준**: 실행 계획의 효율성을 평가하는 체계적 접근법 제공
- **하이브리드 쿼리 분석**: 텍스트와 벡터 조건의 복합 쿼리 최적화 이론의 기초

58.6 현대 DBMS 최적화의 새로운 방향

6.1 머신러닝 기반 최적화의 상태 최근의 발전:

- NeuroCard: 생성 모델을 통한 카디널리티 추정 (정확도 10배 향상)
- Kepler: 강화학습을 통한 플랜 선택
- BayesCard: 베이지안 네트워크로 속성 간 상관관계 학습

한계: - 훈련 데이터의 품질과 양이 매우 중요 - 데이터 분포 변화에 빠르게 적응하기 어려움 - 설명 가능성 (explainability) 부족으로 디버깅 어려움

6.2 적응적 실행(Adaptive Execution) 런타임에서 실행 계획을 동적으로 수정:

- Cardinality feedback: 초기 추정이 틀린 경우 중간에 전략 변경
- Cascade join: 조인 결과의 실제 크기에 따라 다음 조인 방식 선택
- 장점: 단순한 구현으로도 큰 성능 향상 가능
- 단점: 추가 오버헤드와 복잡도

6.3 벡터 통합의 미해결 문제들 - 벡터 인덱스의 재계산 비용이 일반 인덱스와 다름 - 근삿값(approximate) 계산의 정확도 영향 - 메모리 계층의 다양한 특성 반영

6.4 쿼리 최적화의 단계별 프로세스

1단계: 쿼리 파싱

- SQL 쿼리를 내부 표현으로 변환
- 문법 검증

2단계: 논리 계획 생성

- WHERE 절의 필터 순서 결정
- 조인 조건 식별

3단계: 물리 계획 생성

- 각 논리 연산을 물리 연산으로 변환
- 인덱스 활용 여부 결정
- 동시성 제어 방식 선택

4단계: 비용 기반 선택

- 가능한 모든 계획의 비용 추정
- 최소 비용 계획 선택

5단계: 실행

- 선택된 계획 실행
- 런타임에 예상 밖 상황 처리

벡터 검색 통합 시에는 이 각 단계에서 벡터 관련 결정이 추가됨

추가 제기 문제

1. **벡터 특화 카디널리티 추정**: 벡터 검색에서 k 값이 결과 크기를 결정할 때, 이를 전통적 카디널리티 추정 프레임에 어떻게 통합할 것인가?
2. **비용 모델의 차원 의존성**: 벡터 차원이 8에서 1536으로 변할 때 비용 모델은 어떻게 스케일링되어야 하는가?
3. **인덱스 구조의 영향**: B-tree와 HNSW 같은 다양한 인덱스의 비용 특성을 어떻게 단일 비용 모델에 통합할 것인가?
4. **적응적 학습의 한계**: 머신러닝 기반 카디널리티 추정이 데이터 분포 변화에 얼마나 빠르게 적응할 수 있는가?
5. **혼합 워크로드 최적화**: 텍스트, 벡터, 관계형 쿼리가 혼재된 환경에서 일관된 최적화 전략은 가능한가?
6. **런타임 적응의 오버헤드**: 적응적 실행이 주는 성능 향상이 추가 계산 비용을 상쇄할 수 있는가?
7. **정확한 vs 근사값 계산의 최적화**: 벡터 검색의 근사값 결과가 최종 쿼리 정확도에 미치는 영향을 비용 모델에 반영할 수 있는가?